

Sécurité Sentinel-X — failles, mesures et preuves

Oct 8, 2026 · @Ma

Ce document décrit le durcissement du serveur et de la chaîne de communication du projet Sentinel-X, dans le cadre du pentest croisé entre équipes. Pour chaque faille : la **description** et le **risque**, la **remédiation** mise en place (code/configuration), et la **preuve** de son efficacité. Architecture : boîtier ESP8266 → Wi-Fi dédié → broker Mosquitto, API FastAPI, dashboard (Caddy) et vision IA dans Docker sous WSL.

Synthèse

Faille	Risque	Mesure appliquée	Preuve
F1 — Broker MQTT ouvert (anonyme, sans ACL)	N'importe qui sur le Wi-Fi peut publier et commander le boîtier	Authentification obligatoire + ACL par compte	Connexion anonyme refusée (<i>not authorised</i>)
F2 — API FastAPI exposée sur le réseau	Accès direct à l'API sans passer par le dashboard	API liée à 127.0.0.1 (localhost)	Port 8000 fermé depuis le Wi-Fi
F3 — MQTT en clair (interceptable)	Sniffing : lecture des mesures et du mot de passe	Chiffrement TLS/MQTTs (port 8883, CA épinglée)	Wireshark : clair avant / TLSv1.2 après
F4 — Dashboard sans authentification	Accès libre au dashboard et aux commandes	basic_auth Caddy (bcrypt)	401 sans login, 200 avec
Surface — ports internes ouverts au Wi-Fi	Services internes atteignables depuis le réseau	Fermeture des ports inutiles + pare-feu	Seuls 8080 et 8883 ouverts (+ 2222 SSH)

F1 — Broker MQTT ouvert → authentification + ACL

Description / risque : à l'origine, Mosquitto acceptait les connexions **anonymes** (`allow_anonymous true`), sans contrôle de topic. N'importe quel appareil sur le Wi-Fi pouvait s'abonner à toute la télémétrie, injecter de fausses mesures, ou **commander le boîtier** (le désarmer, faire sonner le buzzer...).

Remédiation : connexions anonymes interdites, fichier de mots de passe, et ACL par compte (chaque compte limité à ses topics).

`mosquitto.conf` (extrait) :

```
allow_anonymous false
password_file /mosquitto/config/passwd
acl_file /mosquitto/config/acl
```

`acl` — trois comptes, chacun cloisonné :

```
user esp          # boîtier : publie SA télémétrie, lit SES ordres, rien
d'autre
topic write sentinel/groupe1/edge01/telemetry
topic write sentinel/groupe1/edge01/alert
topic write sentinel/groupe1/edge01/status
topic read  sentinel/groupe1/edge01/cmd
topic read  sentinel/groupe1/edge01/config

user vision       # caméra IA : publie ses alertes, lit la télémétrie
topic write sentinel/groupe1/cam-01/alert
topic read  sentinel/+/+/telemetry

user api          # pont/dashboard : lit tout, seul à pouvoir commander
topic read  sentinel/#
topic write sentinel/+/+/cmd
topic write sentinel/+/+/config
```

Le compte `esp` ne peut donc **pas** envoyer de commande (`cmd` est en lecture seule pour lui) : seul le pont `api` commande le boîtier. Cette ACL couvre aussi le topic `config` (réglages retained), qui n'est modifiable que par `api`.

Preuve — une tentative de désarmement en connexion anonyme est rejetée par le broker :

```
# client anonyme -> publier sur sentinel/groupe1/edge01/cmd
{"action":"disarm"}
Connection Refused: not authorised
```

F2 — API FastAPI exposée sur le réseau → bind 127.0.0.1

Description / risque : l'API (FastAPI, port 8000) était publiée sur **toutes les interfaces**. Un attaquant sur le Wi-Fi pouvait l'interroger directement (ex. `POST /api/v1/commands`) en **contournant le dashboard** et son authentification.

Remédiation : l'API n'est plus publiée que sur `127.0.0.1`. Elle reste joignable par le dashboard (Caddy l'atteint via le réseau interne Docker `api:8000`), mais plus depuis le Wi-Fi. Tout accès externe passe donc obligatoirement par Caddy, qui impose l'authentification.

`docker-compose.yml` (extrait) :

```
api:
  ports:
    - "127.0.0.1:8000:8000"    # avant : "8000:8000" (exposée à tout le
    réseau)
```

Preuve — depuis le réseau Wi-Fi, le port 8000 n'est plus joignable (seul Caddy l'atteint, en localhost) :

```
Test-NetConnection 192.168.137.1 -Port 8000
TcpTestSucceeded : False      # API inaccessible depuis le réseau
```

F3 — MQTT en clair → chiffrement TLS/MQTTS

Description / risque : à l'origine, le boîtier parlait au broker en **MQTT clair** (port 1883). Avec Wireshark, on lisait en clair toute la télémétrie **et le mot de passe MQTT** (transmis dans le paquet CONNECT), ce qui permettait d'usurper le boîtier.

Remédiation : écoute **TLS sur le port 8883** (MQTTS). Le broker présente un certificat signé par la CA du groupe ; le boîtier vérifie ce certificat (CA épinglée, cf. document *Firmware — Sécurité*). Tout le trafic, mot de passe compris, est désormais chiffré en TLS 1.2.

`mosquitto.conf` (extrait) :

```
listener 8883 0.0.0.0
cafile    /mosquitto/config/certs/ca.crt
certfile  /mosquitto/config/certs/server.crt
keyfile   /mosquitto/config/certs/server.key
```

Certificat du serveur :

- Sujet `CN=sentinel.local` ; émetteur `CN=Sentinel-X CA` (auto-signée, interne).
- SAN : IP `192.168.137.1`, DNS `sentinel.local` / `mosquitto` / `localhost`.
- Clé **EC P-256** (prime256v1), signature **ECDSA-SHA256**, validité 10 ans.

Preuve 1 — tests fonctionnels (tous passés) :

```

T1  connexion anonyme                -> refusée (not authorised)
T2  connexion sans TLS sur 8883        -> échec (handshake requis)
T3  connexion authentifiée en TLS (esp) -> OK
T4  boîtier publie sa télémétrie en TLS -> OK (heap ~15,6 ko)

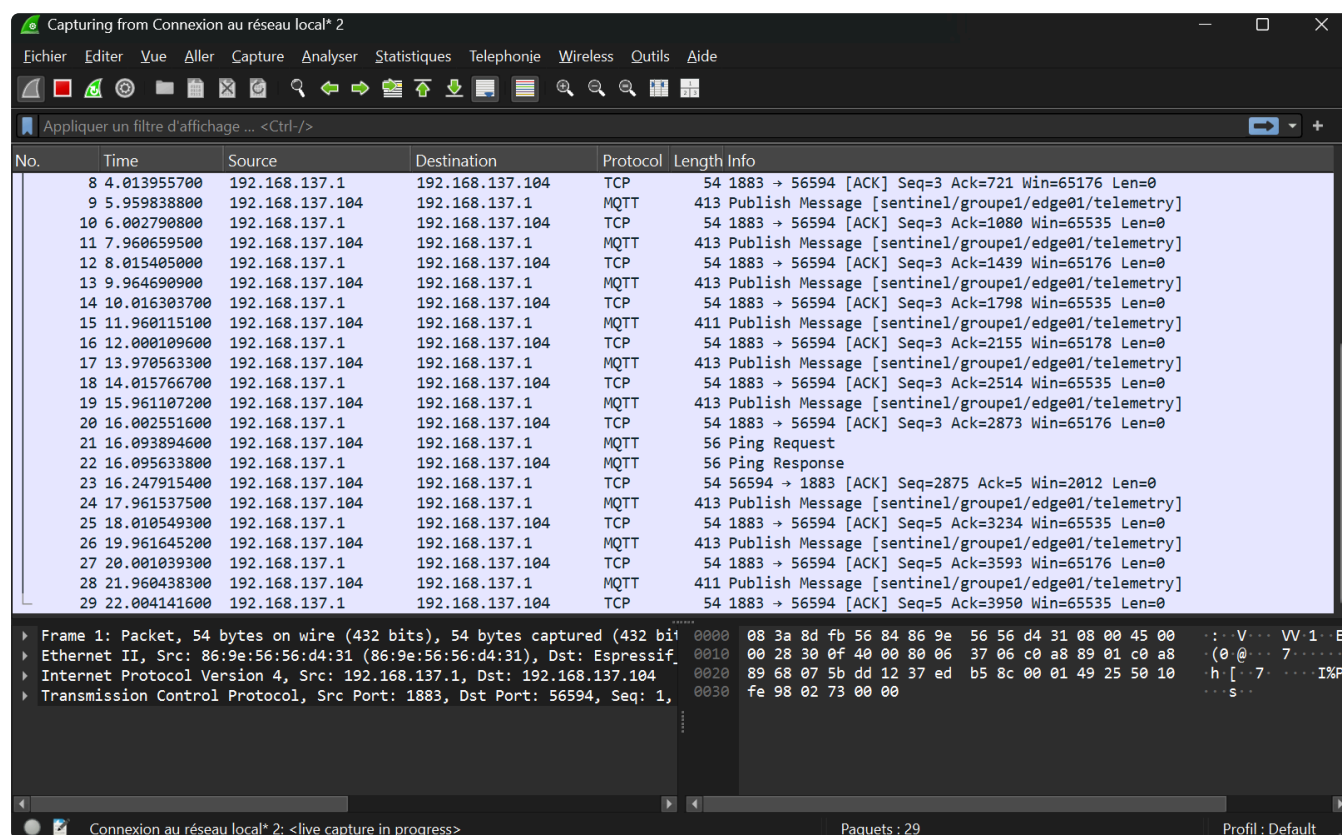
```

Preuve 2 — captures Wireshark :

- *Avant* — filtre `mqtt` : trames MQTT lisibles ; `mqtt.msgtype == 1` (CONNECT) expose identifiant et mot de passe en clair.
- *Après* — filtre `tcp.port == 8883` : uniquement des TLSv1.2 Application Data, contenu illisible.

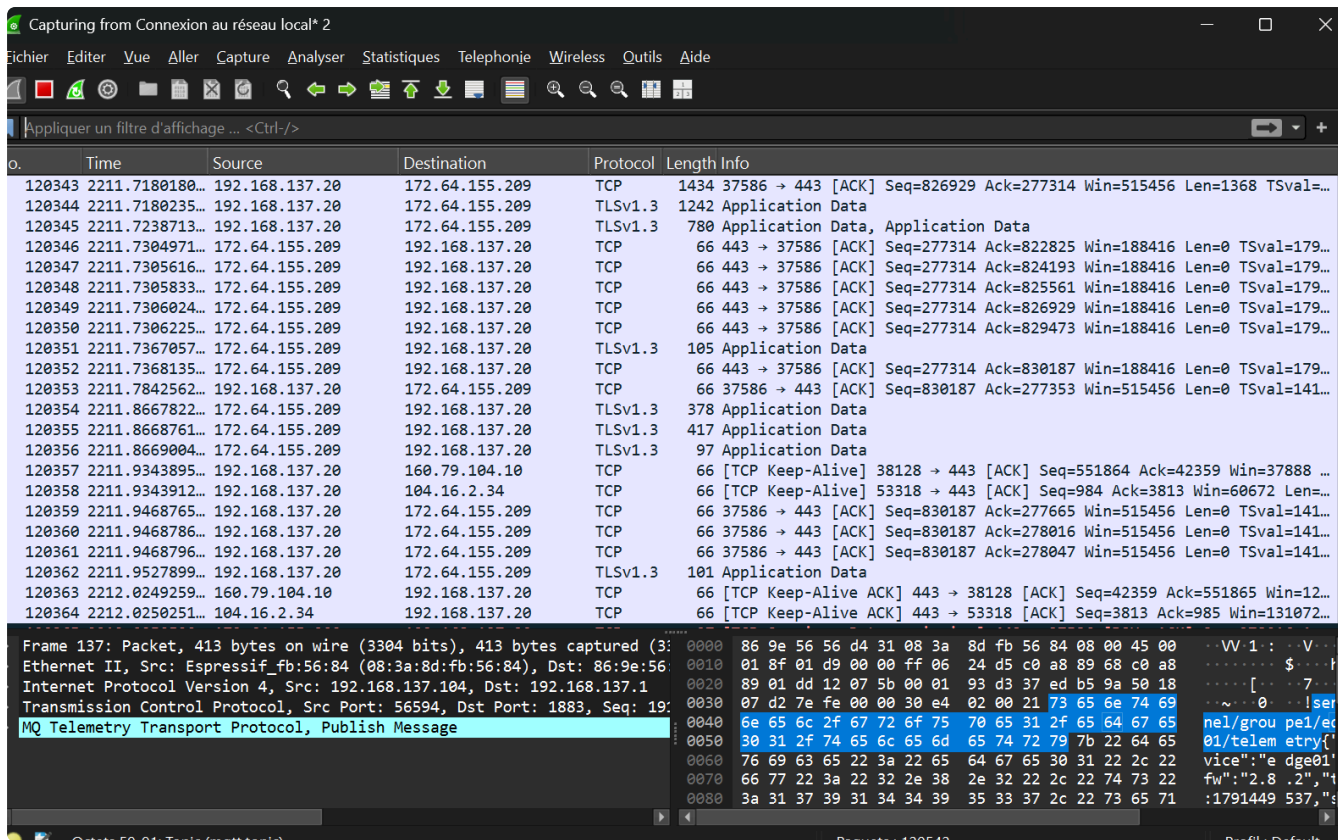
Avant TLS — trafic MQTT en clair sur le port 1883 : le topic

`sentinel/groupe1/edge01/telemetry` est lisible pour chaque *Publish Message*.



En détaillant une trame, la charge utile JSON apparaît en clair

(`"device": "edge01", "fw": "2.8.2"...`) : n'importe qui sur le Wi-Fi peut lire les mesures — et, dans le paquet CONNECT, le mot de passe.



📌 Capture « APRÈS » (TLSv1.2 sur 8883) — à ajouter : avec le filtre `tcp.port == 8883`, le contenu n'apparaît plus qu'en `Application Data` chiffré.

F4 — Dashboard sans authentification → `basic_auth` Caddy

Description / risque : le dashboard — et donc les boutons de commande du boîtier — était accessible **sans aucune authentification** à toute personne sur le Wi-Fi.

Remédiation : Caddy protège l'ensemble du site par `basic_auth` (mot de passe haché en `bcrypt`, coût 14). Aucune page ni aucun endpoint n'est servi sans identifiant valide. Combiné à F2 (API en localhost), Caddy devient le **seul point d'entrée**, et il est authentifié.

Caddyfile (extrait) :

```

:80 {
  basic_auth {
    sentinel $2a$14$...      # hash bcrypt du mot de passe (jamais en
clair)
  }
  handle /api/* { reverse_proxy api:8000 }
  handle /ws*   { reverse_proxy api:8000 }
  handle /video* { reverse_proxy vision:8081 }
  handle        { root * /srv; file_server }
}

```

Preuve — l'accès sans identifiant est refusé (401), avec identifiant il est accepté (200) :

```

sans login : HTTP 401 Unauthorized
avec login : HTTP 200 OK

```

Réduction de la surface d'exposition

Description / risque : plusieurs ports internes étaient redirigés vers le Wi-Fi via `netsh portproxy` (1883 MQTT clair, 8000 API, 443). Ces redirections rendaient joignables depuis le réseau des services qui n'ont pas à l'être.

Remédiation : suppression des redirections `portproxy` inutiles (1883, 8000, 443). Seuls restent exposés au Wi-Fi les ports strictement nécessaires. Les règles du pare-feu Windows sont en outre limitées au sous-réseau du hotspot (`192.168.137.0/24`).

Port	Service	Avant	Après
1883	MQTT en clair	ouvert	fermé
8000	API directe	ouvert	fermé
443	(inutilisé)	ouvert	fermé
2222	SSH (par clé)	ouvert	ouvert
8080	Dashboard (authentifié)	ouvert	ouvert
8883	MQTTS (TLS)	ouvert	ouvert

Preuve — après nettoyage, seuls 2222, 8080 et 8883 répondent depuis le réseau :

2222	OUVERT	(SSH par clé)
8080	OUVERT	(dashboard, authentifié)
8883	OUVERT	(MQTTS)
1883	fermé	(MQTT clair)
8000	fermé	(API directe)

Note : la suppression des redirections `portproxy` doit être lancée dans un PowerShell **administrateur** ; le re-test (`Test-NetConnection`) devient fiable une fois les sockets de redirection retirés.